设计理念

📣 react是javascript构建 **快速响应** 的大型web应用程序的首选方式

什么因素制约快速响应??

大量的计算 网络不佳?

计算能力 网路延迟

cpu瓶颈

1000 / 60Hz js脚本执行 样式布局 样式绘制

https://codesandbox.io/s/concurrent-3h48s?file=/src/index.j

同步更新 => 异步可中断更新

IO瓶颈

将人 交互实现到UI交互

在屏幕之前切换时显示过多的中间加载状态会使切换的速度变慢

基础包结构

1. **react**

> react 基础包, 只提供定义 react 组件(`ReactElement`)的必要函数, 一般来说需要和渲染器(`react-dom`, `react-native`)一同使用. 在编写 `react` 应用的代码时, 大部分都是调用此包的 api.

2. **react-dom**

> react 渲染器之一, 是 react 与 web 平台连接的桥梁(可以在浏览器和 nodejs 环境中使用), 将 `react-reconciler` 中的运行结果输出到 web 界面上. 在编写 `react` 应用的代码时,大多数场景下, 能用到此包的就是一个入口函数 `ReactDOM.render(<App/>, document.getElementById('root'))`, 其余使用的 api, 基本是 `react` 包提供的.

3. **react-reconciler**

> react 得以运行的核心包(综合协调 `react-dom`, `react`, `scheduler` 各包之间的调用与配合). 管理 react 应用状态的输入和结果的输出. 将输入信号最终转换成输出信号传递给渲染器.

- 接受输入(`scheduleUpdateOnFiber`), 将 `fiber` 树生成逻辑封装到一个回调函数中(涉及 `fiber` 树形结构, `fiber.updateQueue` 队列, 调和算法等),

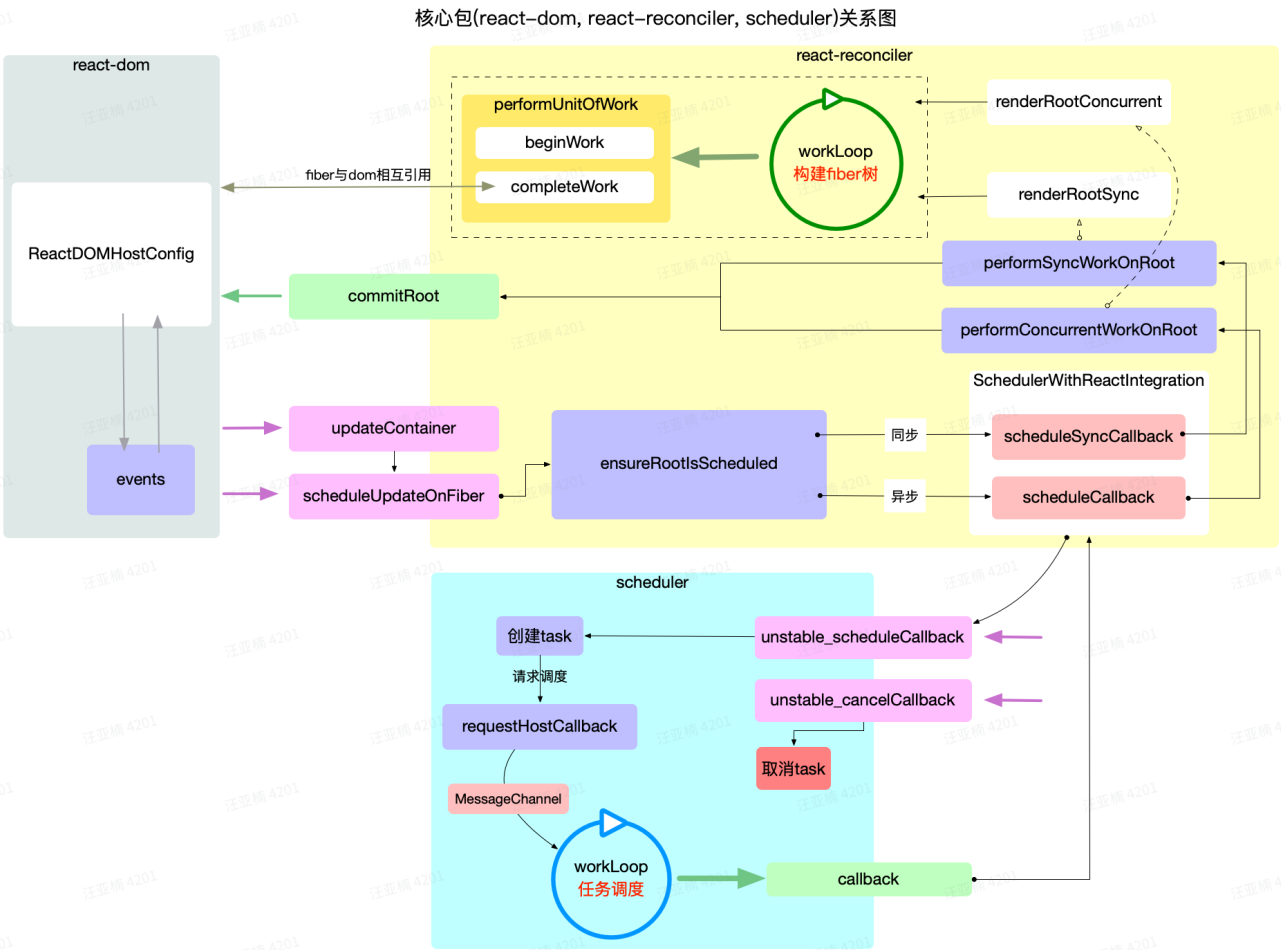
- 把此回调函数(`performSyncWorkOnRoot` 或 `performConcurrentWorkOnRoot`)送入 `scheduler` 进行调度
- `scheduler` 会控制回调函数执行的时机, 回调函数执行完成后得到全新的 fiber 树
- 再调用渲染器(如 `react-dom`, `react-native` 等)将 fiber 树形结构最终反映到界面上

4. **scheduler**

> 调度机制的核心实现, 控制由 `react-reconciler` 送入的回调函数的执行时机, 在 `concurrent` 模式下可以实现任务分片. 在编写 `react` 应用的代码时, 同样几乎不会直接用到此包提供的 api.

- 核心任务就是执行回调(回调函数由 `react-reconciler` 提供)
- 通过控制回调函数的执行时机, 来达到任务分片的目的, 实现可中断渲染(`concurrent` 模式下才有此特性)

两大工作循环



Fiber 纤程

https://github.com/facebook/react/issues/7942#issuecomment-254987818

https://github.com/acdlite/react-fiber-architecture

代数效应 将副作用从函数中剔除

https://codesandbox.io/s/frosty-hermann-bztrp?file=/src/index.js

文章

1. stack reconciler
2. https://github.com/acdlite/react-fiber-architecture
3. https://medium.com/react-in-depth/inside-fiber-in-depth-overview-of-the-new-reconciliation-algorithm-in-react-e1c04700ef6e